

What Makes a Hit

Spotify Audio Feature Analysis Dashboard

Design Document

Navya Neelamegam

School of Information Studies, Syracuse University

April 21, 2026

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
2	Technology Stack	2
3	Project Structure	3
4	Data Model	3
4.1	Dataset Schema	3
4.2	Supported Genres	4
4.3	Data Generation Algorithm	4
4.4	Genre-Specific Feature Distributions	4
5	Dashboard Architecture	5
5.1	Application Flow	5
5.2	Data Source Selection	5
5.3	Sidebar Controls	6
5.4	Tab Layout and Visualizations	6
5.4.1	Tab 1: Overview	7
5.4.2	Tab 2: Correlations	7
5.4.3	Tab 3: Scatter Explorer	7
5.4.4	Tab 4: Genre Comparison	7
5.4.5	Tab 5: Trends Over Time	7
5.4.6	Tab 6: Top Tracks	8
5.4.7	Tab 7: Clustering	8
5.4.8	Tab 8: PCA	8
5.4.9	Tab 9: Predictive Modeling	9

6	Kaggle Integration	10
6.1	Dataset Download via kagglehub	10
6.2	Automatic Column Mapping	11
6.3	Data Normalization Pipeline	11
6.4	Graceful Degradation	11
7	Spotify Web API Integration	12
7.1	Authentication	12
7.2	Retrieval Modes	12
7.2.1	Playlist URL/ID	12
7.2.2	Search Tracks	13
7.2.3	Artist Discography	13
7.3	API Data Pipeline	13
7.4	URL Parsing	14
7.5	Caching and Rate Limits	14
7.6	Security Considerations	14
8	Key Code Design	14
8.1	Data Loading and Caching	14
8.2	Filtering Pipeline	15
8.3	Correlation Analysis	15
8.4	Radar Chart Construction	15
9	Deployment and Execution	16
9.1	Prerequisites	16
9.2	Installation	16
9.3	Data Generation	16
9.4	Running the Dashboard	16
10	Design Decisions and Trade-offs	16
11	Future Enhancements	17

Introduction

This document describes the complete design of the “**What Makes a Hit**” Spotify Audio Feature Analysis Dashboard. The dashboard is an interactive web application built with Streamlit that enables users to explore how audio features such as energy, danceability, valence, acousticness, and others relate to song popularity across genres and time periods.

Purpose

The dashboard answers the question: *What audio characteristics make a song popular?* It provides interactive visualizations that allow users to:

- Explore the distribution of popularity across genres
- Identify correlations between audio features and popularity
- Compare genre-specific audio profiles
- Track feature trends over decades (1990–2024)
- Discover top tracks filtered by custom criteria
- Cluster tracks into natural groupings using K-Means or DBSCAN
- Visualize the audio feature space in 2D/3D via PCA
- Train regression models to predict popularity from audio features
- Pull live data from the Spotify Web API or Kaggle

Scope

The system comprises three components:

1. **Data Generation Module** (`generate_data.py`) — Produces a realistic synthetic dataset
2. **Dataset** (`data/spotify_data.csv`) — 5,000 track records with 14 columns
3. **Dashboard Application** (`app.py`) — Interactive Streamlit web app with 9 analysis tabs

Technology Stack

Table 1: Software Dependencies

Package	Version	Role
Python	3.7+	Runtime environment
Streamlit	1.23.1	Dashboard framework
Pandas	1.3.5	Data manipulation and analysis
Plotly	5.18.0	Interactive visualizations
NumPy	1.21.6	Numerical computations
scikit-learn	1.0.2	Clustering (K-Means, DBSCAN) and PCA

All packages are installed to a user-specified directory (referred to as `<pkg_install_path>` throughout this document) and made available via the `PYTHONPATH` environment variable.

Table 2: Additional Dependencies

Package	Version	Role
kagglehub	0.2.9	Download datasets from Kaggle
spotipy	2.23.0	Spotify Web API client library

Project Structure

```

navya/
  app.py           # Main Streamlit dashboard application
  generate_data.py  # Synthetic dataset generator
  data/
    spotify_data.csv # Generated dataset (5,000 tracks)
  docs/
    design_document.tex # This document (LaTeX source)

```

Data Model

Dataset Schema

The dataset contains 5,000 synthetic track records. Each record has 14 attributes described in Table 3.

Table 3: Dataset Schema (`spotify_data.csv`)

Column	Type	Description
<code>track_name</code>	string	Song title generated from a word pool
<code>artist</code>	string	Artist name selected per genre
<code>genre</code>	string	One of 10 genres (see Section 4.2)
<code>release_year</code>	integer	Year of release (1990–2024)
<code>popularity</code>	integer	Popularity score (0–100)
<code>danceability</code>	float	Suitability for dancing (0.0–1.0)
<code>energy</code>	float	Perceptual intensity and activity (0.0–1.0)
<code>valence</code>	float	Musical positiveness (0.0–1.0)
<code>acousticness</code>	float	Confidence the track is acoustic (0.0–1.0)
<code>instrumentalness</code>	float	Likelihood of no vocal content (0.0–1.0)
<code>speechiness</code>	float	Presence of spoken words (0.0–1.0)
<code>tempo</code>	float	Estimated tempo in BPM (60–220)
<code>loudness</code>	float	Overall loudness in dB (–30 to 0)
<code>duration_ms</code>	integer	Track duration in milliseconds (120k–360k)

Supported Genres

The dataset spans 10 music genres with the following generation probabilities:

Table 4: Genre Distribution in Generated Data

Genre	Probability	Expected Count
Pop	18%	900
Hip-Hop	15%	750
Rock	12%	600
Electronic	10%	500
R&B	10%	500
Latin	10%	500
Indie	8%	400
Country	7%	350
Jazz	5%	250
Classical	5%	250

Data Generation Algorithm

Each track’s audio features are sampled from genre-specific Gaussian distributions. The generation process follows these steps:

1. **Genre assignment:** Randomly selected according to the probability distribution in the table above.
2. **Feature sampling:** For each audio feature f , a value is drawn from $\mathcal{N}(\mu_{g,f}, \sigma_{g,f})$ where g is the genre. Values are clamped to their valid ranges.
3. **Artist assignment:** Randomly selected from a pool of 10 real artists per genre.
4. **Track name generation:** Composed of 1–3 randomly chosen words from a 40-word vocabulary.
5. **Popularity computation:** Calculated using a weighted formula with noise:

The popularity formula is:

$$pop = clamp\left(0.3 \cdot d + 0.2 \cdot e + 0.15 \cdot v + 0.1 \cdot (1 - a) + \frac{y - 1990}{34} \cdot 0.15 + \epsilon, 0, 1\right)$$

where:

- d = danceability, e = energy, v = valence, a = acousticness
- y = release year
- $\epsilon \sim \mathcal{N}(0, 0.12)$ adds realistic noise
- The result is scaled to 0–100 integer range

Genre-Specific Feature Distributions

Table 5 shows the mean (μ) and standard deviation (σ) for key audio features per genre.

Table 5: Genre Audio Feature Parameters (μ, σ)

Genre	Dance.	Energy	Valence	Acoust.	Instrum.	Speech.
Pop	.65, .12	.70, .12	.55, .18	.20, .15	.02, .03	.08, .05
Rock	.50, .13	.78, .12	.45, .20	.15, .12	.05, .07	.05, .03
Hip-Hop	.75, .10	.65, .14	.50, .20	.12, .10	.01, .02	.18, .10
Electronic	.72, .11	.80, .10	.40, .20	.05, .05	.30, .25	.07, .05
R&B	.68, .12	.55, .15	.50, .20	.30, .18	.02, .03	.10, .06
Latin	.78, .10	.72, .12	.65, .18	.18, .14	.02, .03	.09, .05
Country	.55, .12	.60, .14	.58, .20	.40, .20	.01, .02	.04, .03
Jazz	.45, .15	.40, .18	.45, .22	.65, .20	.25, .25	.05, .04
Indie	.52, .14	.58, .16	.42, .22	.35, .22	.10, .15	.05, .04
Classical	.25, .12	.25, .15	.30, .20	.90, .08	.85, .12	.04, .02

Dashboard Architecture

Application Flow

The dashboard follows a reactive data-flow architecture provided by Streamlit:

1. **Data Loading:** CSV is loaded once via `@st.cache_data` for performance.
2. **User Input:** Sidebar widgets collect filter parameters (genres, year range, popularity range).
3. **Data Filtering:** A boolean mask is applied to produce the filtered DataFrame.
4. **Visualization:** Nine tabs render interactive Plotly charts from the filtered data.
5. **Re-execution:** Any widget change triggers a full re-run with cached data.

Data Source Selection

The sidebar begins with a data source selector offering four modes:

Table 6: Data Source Options

Source	Description
Bundled Dataset	Uses the pre-generated <code>data/spotify_data.csv</code> (5,000 tracks). Default option.
Kaggle Dataset	Downloads any public dataset from https://www.kaggle.com/datasets via kagglehub. Accepts a dataset slug (e.g., <code>maharshipandya/spotify-tracks-dataset</code>) or a full Kaggle URL. If the dataset contains multiple CSV files, the user selects which one to load.
Spotify API	Connects to the Spotify Web API in real time via <code>spotipy</code> . Supports three retrieval modes: Playlist, Search, and Artist Discography (see Section 7).
Upload CSV	Drag-and-drop or browse for a local CSV file.

For Kaggle and uploaded datasets, the app applies an automatic column mapping and normalization pipeline (see Section 6.2).

Sidebar Controls

Table 7: Sidebar Filter Widgets

Widget	Type	Description
Genre Filter	Multiselect	Select one or more genres (default: all). Hidden if no genre column detected.
Release Year	Range Slider	Filter tracks by release year. Hidden if no year column detected.
Popularity	Range Slider	Filter by popularity score 0–100 (default: full range). Hidden if no popularity column.
Column Mapping	Expander	Shows which audio features were auto-mapped and which are missing (external datasets only).
Tracks Shown	Metric	Displays count of tracks matching current filters

Tab Layout and Visualizations

The dashboard contains nine tabs, each serving a distinct analytical purpose.

5.4.1 Tab 1: Overview

Provides a high-level summary of the filtered dataset.

- **KPI Cards** (4 columns): Total Tracks, Average Popularity, Average Danceability, Average Energy
- **Popularity Histogram**: Stacked histogram of popularity scores colored by genre (30 bins)
- **Genre Pie Chart**: Proportional breakdown of tracks by genre

5.4.2 Tab 2: Correlations

Reveals relationships between audio features and popularity.

- **Correlation Heatmap**: 9×9 matrix (8 audio features + popularity) using Plotly `imshow` with `RdBu_r` color scale, annotated with correlation coefficients
- **Insight Box**: Automatically identifies and reports the strongest positive and negative correlations with popularity

5.4.3 Tab 3: Scatter Explorer

Enables free-form exploration of feature relationships.

- **Controls**: Three dropdowns for X-axis feature, Y-axis feature, and color variable (genre, popularity, or release year)
- **Scatter Plot**: Interactive Plotly scatter with hover tooltips showing track name, artist, and popularity
- Point opacity set to 0.6 with marker size 5 for readability at 5,000 points

5.4.4 Tab 4: Genre Comparison

Compares audio profiles across genres.

- **Grouped Bar Chart**: Mean values of 0–1 scale features (danceability, energy, valence, acousticness, instrumentalness, speechiness) grouped by genre
- **Radar Chart**: Polar plot overlaying selected genres' feature profiles with fill. User selects which genres to compare via a multiselect widget
- **Horizontal Bar Chart**: Average popularity by genre, color-mapped with Viridis scale

5.4.5 Tab 5: Trends Over Time

Tracks how audio features and popularity have evolved from 1990 to 2024.

- **Feature Trend Lines**: Multi-line chart of user-selected features averaged per release year (default: danceability, energy, popularity)
- **Genre Popularity Trend**: Line chart showing average popularity over time, one line per genre

5.4.6 Tab 6: Top Tracks

A filterable, sortable data table.

- **Sort Selector:** Dropdown to choose the ranking column (default: popularity)
- **Count Slider:** Select how many tracks to display (10–100, default 25)
- **Data Table:** Full-width Streamlit dataframe showing track name, artist, genre, release year, popularity, and all audio features

5.4.7 Tab 7: Clustering

Applies unsupervised machine learning to identify natural groupings of tracks in audio feature space using scikit-learn.

- **Feature Selector:** Multiselect to choose which audio features (and optionally popularity) to include in clustering. Defaults to all 0–1 scale features.
- **Algorithm Toggle:** Radio button to switch between K-Means and DBSCAN.
- **K-Means Controls:** Slider for number of clusters k (2–10, default 4).
- **DBSCAN Controls:** Sliders for epsilon (0.1–3.0, default 0.8) and minimum samples (2–20, default 5). Also displays a noise point count metric.
- **Cluster Scatter Plot:** Interactive Plotly scatter colored by cluster assignment. Users select X and Y axes from the clustering features.
- **Cluster Profiles Table:** Mean feature values per cluster displayed in a formatted DataFrame.
- **Cluster Profile Bar Chart:** Grouped bar chart comparing mean feature values across clusters.
- **Popularity by Cluster:** Box plot showing the popularity distribution within each cluster (shown only when popularity data is available).

Implementation details:

- All selected features are standardized via `StandardScaler` before clustering to ensure equal feature weighting regardless of original scale.
- Rows with missing values in the selected features are dropped before fitting.
- K-Means uses `n_init=10` and `random_state=42` for reproducibility.
- DBSCAN labels noise points as cluster “–1”, which are included in the scatter plot for transparency.

5.4.8 Tab 8: PCA

Applies Principal Component Analysis (PCA) via scikit-learn to reduce the high-dimensional audio feature space to 2D or 3D for visual exploration.

- **Feature Selector:** Multiselect to choose which audio features (and optionally popularity) to include. Defaults to all available audio features. Requires at least 3 features.
- **Projection Toggle:** Radio button to switch between 2D and 3D projections.
- **Color Selector:** Dropdown to color points by genre, popularity, release year, or cluster assignment (if clustering has been run in Tab 7).

- **2D Scatter Plot:** Interactive Plotly scatter of PC1 vs PC2 with hover tooltips showing track name, artist, and popularity. Axis labels include the percentage of variance explained by each component.
- **3D Scatter Plot:** Interactive Plotly 3D scatter of PC1/PC2/PC3 with rotation and zoom controls.
- **Explained Variance Metrics:** KPI cards showing the variance explained by each principal component (PC1, PC2, and optionally PC3).
- **Variance Bar Chart:** Bar chart displaying the variance explained by each component with percentage labels.
- **Feature Loadings Table:** DataFrame showing how each original feature maps onto the principal components, formatted to 3 decimal places.
- **Loadings Heatmap:** Annotated heatmap (RdBu_r color scale, -1 to $+1$) showing the contribution of each original feature to each principal component. This reveals which audio features drive separation along each axis.

Implementation details:

- All selected features are standardized via `StandardScaler` before PCA to prevent features with larger scales (e.g., tempo) from dominating.
- Rows with missing values are dropped before fitting.
- The PCA loadings matrix (`pca_model.components_.T`) maps original features to principal components, enabling interpretation of what each PC represents musically.
- When the clustering tab has been used, cluster labels are available as a color option, enabling visual assessment of whether clusters separate cleanly in reduced-dimensional space.

5.4.9 Tab 9: Predictive Modeling

Trains a regression model to predict track popularity from audio features, providing both evaluation metrics and an interactive prediction tool.

- **Feature Selector:** Multiselect to choose which audio features to use as predictors. Defaults to all available features.
- **Model Selector:** Dropdown with three algorithms:
 - **Random Forest** — ensemble of decision trees with configurable tree count (50–500)
 - **Gradient Boosting** — sequential boosting with configurable tree count (50–500) and learning rate (0.01–0.3)
 - **Linear Regression** — ordinary least squares baseline
- **Train/Test Split:** Slider to configure the test set percentage (10–40%, default 20%).
- **Performance Metrics** (4 KPI cards): R^2 (Train), R^2 (Test), MAE (Test), RMSE (Test).
- **Actual vs Predicted Scatter:** Test-set predictions plotted against actual values with a diagonal reference line. Points near the line indicate accurate predictions.
- **Residual Histogram:** Distribution of prediction errors (actual – predicted) to assess bias and variance.

- **Feature Importance:** Horizontal bar chart showing each feature's contribution to the model. Uses `feature_importances_` for tree-based models and absolute coefficients for linear regression.
- **Interactive Prediction Tool:** Sliders for each audio feature (initialized to dataset means) that feed into the trained model in real time, displaying the predicted popularity score.

Implementation details:

- All features are standardized via `StandardScaler` before training. The scaler is fit on training data only and applied to test data to prevent data leakage.
- Train/test splitting uses `random_state=42` for reproducibility.
- Rows with missing values in selected features or popularity are dropped before splitting.
- A minimum of 20 data points is required to train a model; fewer triggers a warning.
- The interactive prediction tool reuses the same scaler fitted during training, ensuring consistency between evaluation and inference.
- Predicted values are clamped to the 0–100 range for display.

Kaggle Integration

Dataset Download via kagglehub

The dashboard uses the `kagglehub` library to download public datasets directly from Kaggle. Users can provide either:

- A dataset slug: `maharshipandya/spotify-tracks-dataset`
- A full Kaggle URL: `https://www.kaggle.com/datasets/maharshipandya/spotify-tracks-da`

The URL is automatically parsed to extract the slug. Downloads are cached by Streamlit so repeated access is instant.

```
@st.cache_data(show_spinner="Downloading from Kaggle...")
def download_kaggle_dataset(slug):
    import kagglehub
    path = kagglehub.dataset_download(slug)
    return path
```

Listing 1: Kaggle dataset download

Authentication: Kaggle credentials must be configured via one of:

- Environment variables: `KAGGLE_USERNAME` and `KAGGLE_KEY`
- Config file: `~/.kaggle/kaggle.json`

If a downloaded dataset contains multiple CSV files, the user selects which file to load via a sidebar dropdown.

Automatic Column Mapping

External datasets (Kaggle or uploaded) rarely use identical column names. The `auto_map_columns()` function maps common column name variants to the expected schema using a case-insensitive alias table:

Table 8: Column Name Alias Mapping

Target Column	Recognized Aliases
<code>track_name</code>	<code>track_name</code> , <code>name</code> , <code>song_name</code> , <code>song</code> , <code>title</code> , <code>track</code>
<code>artist</code>	<code>artist</code> , <code>artist_name</code> , <code>artists</code> , <code>performer</code>
<code>genre</code>	<code>genre</code> , <code>track_genre</code> , <code>top genre</code> , <code>music_genre</code> , <code>category</code>
<code>release_year</code>	<code>release_year</code> , <code>year</code> , <code>release_date</code> , <code>released_year</code>
<code>popularity</code>	<code>popularity</code> , <code>track_popularity</code> , <code>pop</code> , <code>score</code>
<code>danceability</code>	<code>danceability</code> , <code>danceability_%</code>
<code>energy</code>	<code>energy</code> , <code>energy_%</code>
<code>valence</code>	<code>valence</code> , <code>valence_%</code>
<code>acousticness</code>	<code>acousticness</code> , <code>acousticness_%</code>
<code>instrumentalness</code>	<code>instrumentalness</code> , <code>instrumentalness_%</code>
<code>speechiness</code>	<code>speechiness</code> , <code>speechiness_%</code>
<code>tempo</code>	<code>tempo</code> , <code>bpm</code>
<code>loudness</code>	<code>loudness</code> , <code>loudness_db</code>
<code>duration_ms</code>	<code>duration_ms</code> , <code>duration</code> , <code>length</code>

Data Normalization Pipeline

After column mapping, the `prepare_dataframe()` function applies these transformations:

1. **Percent normalization:** Features with values > 1.0 (e.g., `danceability_%` stored as 0–100) are divided by 100 to produce 0–1 scale values. Tempo and loudness are excluded.
2. **Year extraction:** If the release year column contains full dates (e.g., 2023-05-12), a 4-digit year is extracted via regex. Non-parseable rows are dropped.
3. **Genre fallback:** If no genre column exists, a default "unknown" genre is assigned. Existing genres are lowercased and stripped.
4. **Type coercion:** Popularity is cast to integer; all audio features are coerced to numeric with errors set to NaN.

Graceful Degradation

All visualization tabs adapt to the available columns (applies to all external sources: Kaggle, Spotify API, and uploaded CSVs). If a dataset lacks certain features:

- Missing KPI cards are hidden
- Correlation matrix uses only the features present

- Scatter explorer offers only available features in dropdowns
- Genre comparison and trends tabs show warnings if required columns are absent
- The Column Mapping Info expander reports which features were mapped and which are missing

Spotify Web API Integration

The dashboard connects to the Spotify Web API in real time using the `spotipy` library (v2.23.0). This enables users to pull live track metadata and audio features directly from Spotify without pre-downloading any dataset.

Authentication

The Spotify Web API uses OAuth 2.0 Client Credentials flow, which requires:

- **Client ID** — obtained from the Spotify Developer Dashboard
- **Client Secret** — obtained from the same dashboard

Both are entered in the sidebar as password-masked text inputs. The credentials are passed to `spotipy.SpotifyClientCredentials` to obtain an access token automatically.

```
def get_spotify_client(client_id, client_secret):  
    import spotipy  
    from spotipy.oauth2 import SpotifyClientCredentials  
    auth_manager = SpotifyClientCredentials(  
        client_id=client_id,  
        client_secret=client_secret,  
    )  
    return spotipy.Spotify(auth_manager=auth_manager)
```

Listing 2: Spotify client initialization

How to obtain credentials:

1. Visit <https://developer.spotify.com/dashboard>
2. Create a new application (no redirect URI needed for Client Credentials flow)
3. Copy the Client ID and Client Secret into the dashboard sidebar

Retrieval Modes

The Spotify API data source offers three retrieval modes, each targeting a different use case.

7.2.1 Playlist URL/ID

Loads all tracks from a Spotify playlist. The user provides either:

- A full URL: `https://open.spotify.com/playlist/37i9dQZF1DXcBWIGoYBM5M`
- A Spotify URI: `spotify:playlist:37i9dQZF1DXcBWIGoYBM5M`
- A bare playlist ID: `37i9dQZF1DXcBWIGoYBM5M`

The implementation uses pagination to handle playlists of any length, fetching tracks in batches via `sp.playlist_tracks()` and following `next` links. Audio features are retrieved in batches of 100 via `sp.audio_features()`.

```
results = sp.playlist_tracks(playlist_id)
tracks = results["items"]
while results["next"]:
    results = sp.next(results)
    tracks.extend(results["items"])
```

Listing 3: Playlist track retrieval (simplified)

7.2.2 Search Tracks

Searches Spotify’s catalog using free-text queries. Supports Spotify’s search syntax including field filters:

- `genre:pop year:2023` — pop tracks from 2023
- `artist:Drake` — tracks by Drake
- `track:Blinding Lights` — specific song search

Returns up to 50 results (Spotify API limit per request). A slider allows the user to set the desired result count.

7.2.3 Artist Discography

Retrieves an artist’s full catalog by:

1. Searching for the artist by name (or parsing an artist URL)
2. Fetching up to N albums/singles via `sp.artist_albums()` (configurable, default 20)
3. Extracting all unique tracks from those albums via `sp.album_tracks()`
4. Batch-fetching popularity scores via `sp.tracks()` (batches of 50)
5. Batch-fetching audio features via `sp.audio_features()` (batches of 100)

Track deduplication is performed using a `seen_ids` set to avoid counting the same track from different albums (e.g., deluxe editions). The genre column is set to the artist’s name, enabling comparison when multiple artist datasets are loaded in sequence.

API Data Pipeline

For all three modes, the retrieved data follows the same pipeline:

1. **Metadata extraction:** Track name, artist(s), release date, popularity, and duration are extracted from the Spotify track objects.
2. **Audio feature retrieval:** The 8 standard audio features (danceability, energy, valence, acousticness, instrumentalness, speechiness, tempo, loudness) are fetched via the Audio Features endpoint in batches of 100.
3. **DataFrame assembly:** Metadata and audio features are concatenated column-wise into a single DataFrame.
4. **Normalization:** The standard `prepare_dataframe()` pipeline (Section 6.2) is applied—year extraction from release dates, genre cleanup, and type coercion.

URL Parsing

A `parse_spotify_url()` utility automatically extracts the resource type and ID from various input formats:

Table 9: Supported Spotify Input Formats

Format	Example	Parsed As
Web URL	<code>https://open.spotify.com/playlist/ABC123</code>	playlist, ABC123
Spotify URI	<code>spotify:artist:XYZ789</code>	artist, XYZ789
Bare ID	<code>37i9dQZF1DXcBWIGoYBM5M</code>	(inferred), ID

Query parameters (e.g., `?si=...`) are automatically stripped from URLs.

Caching and Rate Limits

All Spotify API fetch functions are decorated with `@st.cache_data(ttl=600)`, caching results for 10 minutes. This:

- Prevents redundant API calls during Streamlit re-runs triggered by filter changes
- Respects Spotify's rate limits (the Client Credentials flow has generous limits for read-only access)
- Automatically refreshes data after the 10-minute TTL expires

Security Considerations

- Client ID and Client Secret are entered via `type="password"` inputs, masking them in the UI
- Credentials are never stored to disk or logged
- The Client Credentials flow does not access any user-specific data—it only reads public catalog information
- Credentials are passed directly to `spotipy` and never transmitted elsewhere

Key Code Design

Data Loading and Caching

Data is loaded once and cached across Streamlit re-runs. The bundled dataset uses a simple cached loader:

```
@st.cache_data
def load_bundled_data():
    return pd.read_csv("data/spotify_data.csv")
```

Listing 4: Cached data loading

Kaggle downloads are also cached to avoid redundant network requests:

```
@st.cache_data(show_spinner="Downloading from Kaggle...")
def download_kaggle_dataset(slug):
    import kagglehub
    path = kagglehub.dataset_download(slug)
    return path
```

Listing 5: Cached Kaggle download

Filtering Pipeline

A boolean mask dynamically combines only the filters whose columns are present:

```
mask = pd.Series(True, index=df.index)
if selected_genres and "genre" in df.columns:
    mask = mask & df["genre"].isin(selected_genres)
if year_range is not None and "release_year" in df.columns:
    mask = mask & df["release_year"].between(*year_range)
if pop_range is not None and "popularity" in df.columns:
    mask = mask & df["popularity"].between(*pop_range)
filtered = df[mask]
```

Listing 6: Adaptive data filtering

Correlation Analysis

The correlation matrix is computed on filtered data and visualized as an annotated heatmap:

```
corr = filtered[audio_features + ["popularity"]].corr()
fig = px.imshow(
    corr, text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1, zmax=1,
)
```

Listing 7: Correlation heatmap

Radar Chart Construction

Genre profiles are rendered using Plotly's Scatterpolar trace:

```
for g in selected_genres:
    vals = genre_means.loc[g, features].tolist()
    vals.append(vals[0]) # close the polygon
    fig.add_trace(go.Scatterpolar(
        r=vals,
        theta=features + [features[0]],
        fill="toself", name=g, opacity=0.6,
    ))
```

Listing 8: Radar chart for genre comparison

Deployment and Execution

Prerequisites

- Python 3.7 or later
- All dependencies installed to <pkg_install_path>

Installation

```
pip install --target=<pkg_install_path> \
    streamlit pandas plotly numpy scikit-learn kagglehub spotipy
```

Listing 9: Package installation

Data Generation

```
setenv PYTHONPATH <pkg_install_path>
python generate_data.py
```

Listing 10: Generate synthetic dataset

This produces `data/spotify_data.csv` with 5,000 records.

Running the Dashboard

```
setenv PATH <pkg_install_path>/bin:${PATH}
setenv PYTHONPATH <pkg_install_path>
cd <project_directory>
streamlit run app.py
```

Listing 11: Launch the dashboard

The dashboard will be available at `http://localhost:8501` by default.

Design Decisions and Trade-offs

1. **Synthetic vs. Real Data:** A synthetic dataset was chosen to avoid API key requirements and Kaggle download dependencies. The genre-specific Gaussian distributions produce realistic feature relationships.
2. **Popularity Formula:** The weighted linear model with noise ensures meaningful correlations exist in the data while maintaining realistic variance. Danceability has the highest weight (0.3) reflecting real-world trends.
3. **Plotly over Matplotlib:** Plotly was selected for its native interactivity (hover, zoom, pan) which integrates seamlessly with Streamlit's `plotly_chart` component.
4. **Caching Strategy:** `@st.cache_data` ensures the CSV is read only once per session, providing fast re-renders when filters change.
5. **Tempo and Loudness Exclusion from Radar:** These features use different scales (BPM and dB) than the 0–1 normalized features, so they are excluded from the radar chart to maintain visual consistency.

6. **Year Range Default:** The default year filter starts at 2000 (not 1990) to focus on the most data-rich and relevant period while still allowing exploration of older tracks.

Future Enhancements

- **Export Functionality:** Add download buttons for filtered data and chart images